CLAUDE.md

Working title: GRAFT (placeholder)

A single-player first-person shooter for Android, played with a controller. Military science fiction. The player is a heavily augmented soldier fighting unaugmented human soldiers. The owner works from a phone only. Claude Code does the engineering and makes the assets.

Current phase: prototype. The goal is to get the core systems working and feeling right: gunplay, movement, the damage simulation and the augment system. Story, world, progression and release plans are not decided. Don't build around assumptions about them.

Items marked *(proposal)* are Claude's suggestions and can change. Everything else was decided by the owner.

---

1. Working rules for Claude Code

First session

1. Move sections 3–6 (the game design) into `docs/design.md` and leave a one-line pointer here. This file loads every session, so keep it short.
2. Scaffold the repository (§9.3).

3. Add the tooling: `.mcp.json`, `.claude/settings.json`, scripts in `tools/`, project skills (§10).
4. Add the CI workflow (§11).
5. Check every tool in §10.4. Report what works and what doesn't.
6. Start milestone M0 (§12).

Commit after each step.

Always

- The owner only has a phone. They can't run commands, open an editor or read long diffs. Show progress with screenshots, test results, CI runs and APK builds.
- Don't say something works without evidence: a passing test, a screenshot you looked at, a CI log, or the owner's playtest notes.
- You make the assets (§7). Build each mechanic with simple shapes first so gameplay work isn't blocked, then replace them with finished assets.
- Tuning values (weapons, parts, enemies, gore, camera feel) go in `.tres` resource files, not in code.
- Typed GDScript. Treat static-typing warnings as errors.
- Paid APIs (Meshy, ElevenLabs): ask the owner first and say roughly how many calls you'll make.
- Don't push git tags. The cloud GitHub proxy rejects them; CI creates release tags.
- When a decision changes, update the docs in the same commit.
- Tone: grounded military science fiction. Every ability comes from hardware. No magic, psychic powers, occult or religious-

supernatural elements, and no anime styling.

---

2. The game

- **Player and enemies.** You are augmented. For now, enemies are ordinary unaugmented soldiers who come in waves. The difference between you and them should show everywhere: speed, strength, durability, perception, what you can carry, and how they react to you. It's a power fantasy, but fights should still take skill.
- **Simulation.** Simulate as much as an immersive sim would (bodies, damage, ballistics, enemy behavior), but keep combat fast. Every system has to work at full speed, without menus or pauses.
- **Gunplay.** The goal is the best gunplay in any mobile shooter. References: Call of Duty for gun handling, Doom for speed and aggression, Halo for readable weapons and enemies, F.E.A.R. and Trepang2 for slow motion and hit reactions, S.T.A.L.K.E.R. for weapon weight, Soldier of Fortune for locational damage and gore.
- **Tone.** Warhammer 40,000's hardware and industrial scale without its supernatural side. Deus Ex's augmentation. Halo's military hardware. Soldier of Fortune's grounded, violent military action.
- **Visuals.** Realistic enough that weapons and violence look convincing. Beyond that, the look relies on color, lighting, atmosphere and effects more than on detailed geometry and textures.

- **Not decided yet:** story, world, factions, mission structure, progression, release platform.

---

3. Augmentation

3.1 No classes

There are no classes or archetypes. A character is the set of parts installed in their body, plus their gear. What a character can do comes from how those parts combine.

3.2 Parts are physical, and they interact

Every part is a physical component of the body. That is what lets combinations work without scripting each pair.

Each part has:

- a body location and mounting points (it can require a mount or provide one)
- mass
- power draw or power output
- heat output or heat removal
- data links: values it reads from or sends to other parts
- stat changes and behavior hooks
- a visible model and the changes it makes to the body

Parts affect each other through these shared quantities. For example:

- A reinforced skeleton raises carrying capacity and recoil tolerance. That's what lets you fire a belt-fed machine gun from the hip.
- Muscle actuators raise speed and jump height, but they draw power and make heat, so a cooling part changes how long you can sprint.
- An optics package that tracks targets can feed a fire-control link in the hand, which changes how the gun handles, but only when both are installed.
- Heavy armor adds mass, which shortens wall-runs and changes slide distance.

New parts should plug into these quantities instead of adding special cases. When two parts do something useful together that nobody scripted, the system is working.

(proposal) Parts are also part of the damage model: hits to a body location can damage the parts installed there, and EMP shuts down powered parts. Try this in M2.

3.3 Passive parts

- Chosen before a session. Always on.
- Technical hardware names, for example "NW-4 Nanoweave Fiber Vest".
- They change stats and behavior, and they change how you look: your arms and hands in first person, your body in shadows and reflections, and your gear.

3.4 Active parts

Assigned to buttons. Pressing one switches you into a separate control mode with its own inputs, camera and HUD. Each has visible hardware and a deploy animation.

The owner's examples:

- **Shoulder-launched suicide drone.** A pod on your shoulder opens and launches the drone. You fly it from its camera and detonate it.
- **Monofilament whip.** Hold LT to stop looking around; the right stick then moves the whip through the air. A melee weapon.
- **Time slowdown.** Heavy, F.E.A.R. and S.T.A.L.K.E.R.-style impacts, especially with a SPAS-12. Hits sometimes knock enemies into ragdoll.

3.5 Starter parts *(proposal)*

Part	Location	Effect	Cost or drawback	Appearance
NW-4 Nanoweave Fiber Vest	Torso	Resists small-arms fire	Adds mass	Woven fiber visible at collar and cuffs
Ceramic Trauma Plating	Torso	Large armor pool; stops rifle rounds	Heavy	Bolted plates over chest and shoulders
Titanium-Lattice Skeleton	Skeleton	Higher carry capacity and recoil	Adds mass	Anchor bolts at wrists and knuckles

Part	Location	Effect	Cost or drawback	Appearance
		tolerance; no fall damage		
Electroactive Myofibrils	Muscle	Faster sprint, higher jump	Power draw, heat	Synthetic muscle bands through forearm vents
Synaptic Accelerator	Nervous system	Faster aiming, reloads and recoil recovery	Power draw	Port at the base of the skull
Vestibular Stabilizer	Inner ear	Full accuracy while sliding or wall-running	—	Horizon line in the HUD
Ocular Ballistics Suite	Eyes	Shows penetration paths and headshot zones	Vulnerable to flashbangs	Optic glow in reflections; reticle overlay

Part	Location	Effect	Cost or drawback	Appearance
Fire-Control Link	Hands	Less recoil and faster follow-up shots with linked weapons	Shut off by EMP	Contact pads on the palm; readout on the gun
Dorsal Load Frame	Spine	Hip-fire heavy weapons at full speed	No wall-running	Spinal frame; ammo feed over the shoulder
Subdermal Heat Sinks	Skin	Faster cooling	Hot signature; easier to spot	Fins along the forearms; heat shimmer
Hemostatic Nanites	Blood	Slow healing out of combat; less bleeding	High capacity cost	Wounds visibly close
Kinetic Dampers	Legs	Quiet footsteps, longer slides,	—	Piston joints at knees and ankles

Part	Location	Effect	Cost or drawback	Appearance
		softer landings		

More actives *(proposal)*: mark several targets in slow motion and fire at all of them; shoulder ram; brace mode for heavy weapons; grapple line; through-wall scan; take over a turret or drone.

Shared resources *(proposal)*: power, heat, mass, and a capacity budget for the loadout.

---

4. Test loadouts

The owner described two combinations. They are benchmarks for the systems, built from the same pool of parts as any other loadout. They are not classes.

4.1 Fast pistol loadout (build first)

- **Weapon:** suppressed FN Five-seveN, 5.7×28mm, 20-round magazine, long suppressor. Quiet, precise, fast follow-up shots.
- **Parts:** myofibrils, synaptic accelerator, vestibular stabilizer, kinetic dampers, fire-control link. Actives: whip and time slowdown.
- **Movement:** sprint, slide, wall-run, mantle, air dash.
- **What it should do:** move very fast, land headshots quickly and work through a crowd. Rounds can pass through one enemy into the next.

- **Test** *(proposal)*: 12 enemies in a two-level room. A skilled player clears it in under about 25 seconds without stopping.

4.2 Armored machine-gun loadout

- **Weapon:** belt-fed machine gun modeled on the MG42, about 1,200 rounds per minute, built for an augmented carrier. Barrel heat and barrel changes.
- **Parts:** trauma plating, titanium skeleton, dorsal load frame, heat sinks. Actives *(proposal)*: shoulder ram, brace mode.
- **What it should do:** move slowly, absorb a lot of fire, and tear through cover and enemies with sustained fire.
- **Test** *(proposal)*: two squads behind mixed cover in a warehouse.

4.3 Later

SPAS-12 shotgun with time slowdown: clearing rooms in slow motion, with ragdolls.

---

5. Combat simulation

5.1 Locational damage

Soldier of Fortune's GHOUL engine split bodies into 26 damage zones that each reacted differently. Use the same approach.

- Hitboxes on bones, about 20–26 zones: head, neck, upper and lower torso, groin, upper and lower arms, hands, thighs, shins, feet, each side separate.

- Each zone has its own reactions. Leg hits cause stumbling or limping. Arm and hand hits ruin aim or drop the weapon. Gut hits double enemies over. Head and neck hits kill or incapacitate.
- Shooting a weapon out of an enemy's hand can make them surrender.
- Wounded states: crawling, clutching a wound, calling for help.
- Armor: helmets and plates with protection levels. Penetration against protection decides whether a round gets through.
- Dismemberment from heavy rounds, explosions, the whip and the ram.

5.2 Ballistics

- Fast rounds are hitscan with penetration. Slow objects (grenades, the drone, the whip tip) are physical.
- Weapon data: damage, falloff, penetration, impulse, spread, recoil pattern.
- Material table for penetration: flesh, wood, drywall, sheet metal, concrete, armor plate. Penetration drops with each surface.

5.3 Feedback on every shot

- Camera kick and visible gun recoil, both spring-driven
- Muzzle flash and a brief light
- Layered sound: mechanism, shot, room tail (the tail changes with room size)
- Shell or link ejection
- Hit marker and sound, with separate headshot and kill sounds
- A hit reaction on every hit

- Impact decal and particles by surface
- A few frames of hit-stop on heavy hits and headshots
- Controller rumble where Android allows it
- Cancelable reloads; separate tactical and empty reloads

5.4 First-person animation

Weapon and arm motion is procedural: springs for sway, bob, recoil, landing, turning inertia, sprint, and slide and wall-run tilt, blended with a few authored poses. Spring values live in weapon resources and in an on-device tuning menu, so the owner can tune by feel and send the numbers back.

5.5 Ragdolls and gore

- Partial ragdoll for hit reactions. Godot's `PhysicalBoneSimulator3D` can simulate selected bones and blend them with animation.
- Full ragdoll on death, with impulse applied at the hit bone and scaled by the weapon.
- Blood: entry spray, exit spray when a round passes through, bigger bursts on headshots, splatter on the wall behind the target, pools under bodies, wound decals that stay on the body.
- Dismemberment: pre-split meshes with caps; gibs from a pool.
- Gore setting: off, standard, maximum.
- Everything pooled and capped (§8).
- Slow motion has to hold up: blood, ragdolls and debris still look right at 25% speed.

5.6 Enemy behavior

- Awareness from sight, sound (including suppressed shots at close range) and bodies they find.
- Near misses suppress them: worse accuracy, more time in cover.
- Morale drops with casualties and with seeing an augment up close. When it breaks they flee, freeze, surrender or fire blindly.
- Callouts that react to what you do ("Augment!", "Fall back!").
- Squads flank and suppress together. Only a few enemies shoot at the player at a time.
- Roles *(proposal)*: rifleman, shotgunner, shield carrier, machine gunner, marksman, launcher, anti-augment specialist (EMP grenades, armor-piercing rounds, drone jammer), officer.

---

6. Art direction

Decided: see "Visuals" in §2. No anime or cel shading.

(proposal)

- Detail goes first to the first-person arms, weapons and augment hardware, since they're always on screen. Then hit effects and gore, then lighting, then enemies, then environments.
- Baked lighting with a few dynamic lights. Height fog, dust, smoke, sparks, emissive lights, color grading.
- Effects built from meshes with scrolling textures, not huge particle counts.
- Heavy, worn, industrial hardware. Explore palettes with test renders in M4.

7. Assets

The owner doesn't make assets. Claude makes them in the cloud session.

Blender 5.2 LTS, installed by the setup script and driven by Python. Every asset has a generator script in `raw_assets/`, so it can be rebuilt and adjusted.

- Weapons and augment hardware: hard-surface modeling with modifiers (bevel, boolean, mirror, array, weighted normals), built to real dimensions.
- Environment kits and props: modular pieces on a grid.
- Materials: procedural shaders baked to texture maps with Cycles on the CPU, then compressed for mobile.
- LODs, collision meshes, correct scale and pivots, glTF export.
- After each change, render a preview (turntable or four views) and look at it before committing. Send the important ones to the owner.

Humans: MPFB (MakeHuman for Blender, a free extension) generates rigged human bodies. Check and record the license of each asset pack you use. For animation, start from Quaternius's Universal Animation Library (CC0), retargeted, plus procedural layers for aiming, hit reactions and ragdoll.

Textures and HDRIs: Poly Haven and ambientCG, both CC0.

Sound: layered and processed. Use CC0 sources where they exist. ElevenLabs sound generation needs the owner's approval because it

costs money.

Optional, paid: Meshy (text or image to 3D) for props and starting meshes.

Log every third-party file and its license in `CREDITS.md`.

---

8. Target device and performance

Moto G 5G (2025): MediaTek Dimensity 6300, Mali-G57 MC2 GPU, 4 GB RAM, 720×1604 screen at 120 Hz.

Starting budgets. These are guesses: measure them on the phone in M0 and replace them.

Budget	Start
Frame rate	60 fps; locked 30 as a fallback setting
Render scale	0.8–1.0 of 720p
Draw calls	≤ 150
Triangles on screen	≤ 250k
Active enemies	≤ 12
Ragdolls simulating	≤ 3, frozen after about 2.5 s
Blood decals	≤ 48, oldest fade out
Live particles	≤ 1,500
Dynamic lights	≤ 4; only one casts shadows

Budget	Start
RAM	≤ 1.2 GB
Heat	No throttling below target after 20 minutes

For the machine gun at 1,200 rpm: pooled hitscan, a tracer every few rounds, one shared muzzle light, a cap on decals per second, and a looping fire sound.

---

9. Tech

9.1 Stack

- Godot 4.7.2, the latest stable release as of 2026-11-01. 4.8 is in beta; consider moving once it's stable.
- Mobile renderer (Vulkan). Compare it with the Compatibility renderer in M0 by measured frame time.
- Typed GDScript, Jolt physics, GUT for tests.
- Android arm64, landscape, gamepad required.

9.2 Architecture *(proposal)*

- **Parts:** each part is a resource with the properties in §3.2. A body model adds up mass, power, heat and capacity. Stats are base values plus modifiers from parts. Data links connect parts that read and write shared values.
- **Appearance:** each part lists meshes and attachment points for the first-person arms and the full body. Changing the loadout rebuilds the character model.

- **Input contexts:** base FPS controls sit at the bottom of a stack. Each active part pushes its own control context and pops it when it ends.
- **Active part states:** idle, deploying, active, recovering, cooldown. Handles power and heat costs, interruptions (EMP, damage), and handing over the camera and HUD.
- **DamageInfo:** amount, type, penetration, impulse, zone, bone, point, normal, source, flags.
- **Time:** a global time scale for slowdown, with a separate scale for the player. Audio pitch follows.
- **Pools** for projectiles, decals, blood, gibs, sounds and ragdolls.
- **One event bus** autoload for combat events.

9.3 Repository layout

```
project.godot        # the repo root is the Godot project
root
addons/              # GUT and other addons
src/
  core/              # event bus, pools, input contexts,
settings
  body/              # parts system, stats, appearance
  player/            # movement, camera, first-person
arms
  weapons/           # weapons, ballistics, feedback
  augments/active/   # one folder per active part
  ai/                # perception, squads, morale
  gore/              # damage zones, blood, decals,
ragdolls, dismemberment
  vfx/ audio/ ui/ levels/
```

```
data/                # .tres resources: weapons, parts,
enemies, surfaces, gore
assets/              # exported models, textures, audio
raw_assets/          # Blender generator scripts and
sources (.gdignore)
tests/               # GUT tests
tools/               # setup, check, screenshot scripts
(.gdignore)
docs/                # design, playtest log, credits
notes (.gdignore)
.claude/skills/  .claude/settings.json  .mcp.json
.github/workflows/
```

Put an empty `.gdignore` file in `raw_assets/`, `tools/` and `docs/` so Godot doesn't import them.

9.4 Default controls (remappable)

Input	Action
Left stick / L3	Move / sprint
Right stick / R3	Look / melee
LT / RT	Aim / fire
LB / RB	Active parts 1 and 2
A	Jump (wall-run on contact)
B	Slide / crouch
X	Reload (hold to interact)
Y	Switch weapon (hold to change MG barrel)

Input	Action
D-pad	Active part 3, items
Menu / View	Pause / body status

Aiming: response-curve presets, acceleration on full tilt, radial deadzones, aim assist in layers (slowdown near targets, slight pull while aiming down sights). Gyro aiming from the phone if the controller holds it.

---

10. Cloud setup and tooling

10.1 Environment (the owner sets this once at claude.ai/code)

Cloud sessions read this repo's `CLAUDE.md`, `.mcp.json`, hooks in `.claude/settings.json`, and `.claude/skills/`. They don't install plugins listed in settings. See the cloud environment docs.

1. **Network access:** Custom. Tick the option to include the default list, then add:

```
download.blender.org
extensions.blender.org
static.makehumancommunity.org
api.polyhaven.com
dl.polyhaven.org
ambientcg.com
opengameart.org
```

Add `api.meshy.ai` and `api.elevenlabs.io` only if you use those services. GitHub, npm, PyPI, Ubuntu packages and Docker Hub are already in the default list.

2. **Setup script:**

```
#!/bin/bash
# Runs as root before Claude starts. Must exit 0.
Cached if it finishes in about 5 minutes.
GODOT_VERSION="${GODOT_VERSION:-4.7.2}"
BLENDER_SERIES="${BLENDER_SERIES:-5.2}"
BLENDER_VERSION="${BLENDER_VERSION:-5.2.3}"
ERR=/opt/setup-errors.log

apt-get update -qq || true
apt-get install -y -qq xvfb xauth mesa-vulkan-drivers
libvulkan1 libgl1 libegl1 \
  libxi6 libxxf86vm1 libxfixes3 libxrender1 libsm6
libxkbcommon0 libxcursor1 \
  libxinerama1 libxrandr2 libasound2t64 fontconfig
unzip xz-utils || echo "apt install failed" >> "$ERR"

# Godot editor (Linux, runs headless in the session)
if [ ! -x /opt/godot/godot ]; then
  mkdir -p /opt/godot
  curl -fsSL -o /tmp/godot.zip \
    "https://github.com/godotengine/godot-
builds/releases/download/${GODOT_VERSION}-
stable/Godot_v${GODOT_VERSION}-
stable_linux.x86_64.zip" \
    && unzip -q -o /tmp/godot.zip -d /opt/godot \
    && mv "/opt/godot/Godot_v${GODOT_VERSION}-
stable_linux.x86_64" /opt/godot/godot \
```

```
    && chmod +x /opt/godot/godot \
    || echo "Godot download failed" >> "$ERR"
  rm -f /tmp/godot.zip
fi
ln -sf /opt/godot/godot /usr/local/bin/godot

# Blender (Linux build, used in background mode)
if [ ! -x /opt/blender/blender ]; then
  mkdir -p /opt/blender
  curl -fsSL
"https://download.blender.org/release/Blender${BLENDER
_SERIES}/blender-${BLENDER_VERSION}-linux-x64.tar.xz"
\
    | tar -xJ --strip-components=1 -C /opt/blender \
    || echo "Blender download failed" >> "$ERR"
fi
ln -sf /opt/blender/blender /usr/local/bin/blender
exit 0
```

3. **Environment variables:** only if you use paid services: `MESHY_API_KEY`, `ELEVENLABS_API_KEY`. Anyone who can use the environment can read these. Network secrets in the same dialog keep keys hidden from the session.

If something fails:

- The script logs failures to `/opt/setup-errors.log`, and the SessionStart hook prints them.
- If a GitHub download is blocked inside the session, do the download in a GitHub Actions workflow (Actions has open internet access) and commit the result or attach it to a release.

- If the script takes longer than about 5 minutes, the environment isn't cached. Move the Blender download into the SessionStart hook.

Cloud VMs have about 4 CPU cores, 16 GB RAM, 30 GB disk and no GPU. Android exports run in GitHub Actions.

10.2 MCP servers

`.mcp.json`:

```
{
  "mcpServers": {
    "godot": {
      "command": "xvfb-run",
      "args": ["-a", "-s", "-screen 0 1280x720x24",
"npx", "-y", "@beremaran/godot-agent-loop@2"],
      "env": { "GODOT_PATH": "/opt/godot/godot" }
    }
  }
}
```

- **Godot Agent Loop** (releases) runs the game, takes screenshots, reads the scene tree, sends input, waits for conditions and runs tests. Its developer tests it on Linux under Xvfb with software rendering. 2.0.0 (August 2026) was the latest version checked; use the newest 2.x that works.
- Add Meshy (`npx -y @meshy-ai/meshy-mcp-server`) and ElevenLabs (`uvx elevenlabs-mcp`) entries only after their API keys are set. If an `.mcp.json` variable is unset, Claude Code can silently drop that server (issue).

10.3 Blender tooling

Claude runs Blender directly: `blender -b --factory-startup --python script.py -- <args>`. That covers modeling, materials, baking, rendering and export, so no Blender MCP is needed to start.

The official Blender Lab MCP server (v1.0.3, needs Blender 5.1 or newer) works through a Blender add-on and mainly targets an open Blender window. Its background-mode support is limited. Try it later if direct scripting hits a limit.

10.4 Verification

- `tools/check.sh`: import the project, validate scripts, run GUT (`godot --headless -s addons/gut/gut_cmdln.gd -gdir=res://tests -gexit`).
- `tools/screenshot.sh`: run a scene under Xvfb with software Vulkan and save PNGs. Budget about 30 seconds per shot. Use fixed camera positions in each test room.
- `godot --doctool <dir>` dumps the class reference for the installed version. Check APIs there instead of relying on memory.
- `blender --version` and a test render, to confirm Blender works.
- Godot Agent Loop scenarios: scripted input with assertions (enemy dead, ragdoll active, decal count under the cap).
- On the phone: a debug overlay (FPS, frame time, draw calls, memory, counts of enemies, ragdolls and decals); a bug-report button that saves logs, settings and loadout to a file the owner can share; input recording.

10.5 Skills

- Copy the relevant GodotPrompter skills (MIT, by jame581) into `.claude/skills/`: architecture, state machines, navigation, animation, input, performance, code review. Keep the license file. (overview)
- Write project skills in M0: `godot-verify`, `blender-assets`, `weapon-feel`, `gore-and-ragdolls`, `perf-budget`, `asset-sources`.

10.6 SessionStart hook

`tools/session_start.sh`: exit unless `CLAUDE_CODE_REMOTE=true`. Print the Godot and Blender versions and the contents of `/opt/setup-errors.log`. Import the project if `.godot/` is missing.

---

11. CI (GitHub Actions)

- Runs on pushes to `main` and on manual dispatch.
- Steps: download and cache Godot 4.7.2 and its export templates; import; run GUT and stop on failure; export a debug APK (arm64) signed with a keystore from repository secrets; publish it to a rolling `dev-latest` GitHub Release that the workflow creates.
- Run an explicit import before exporting. Headless exports have dropped resources in past Godot versions.
- Claude checks runs with REST commands (`gh run list`, `gh api repos/{owner}/{repo}/actions/runs`). GraphQL commands such as `gh pr` don't work through the proxy.
- Keep the release keystore in GitHub Secrets, and the owner keeps an offline backup.

---

12. Milestones

Each milestone ends with an APK on the owner's phone and an entry in `docs/playtest-log.md`.

1. **M0 – Foundation.** Repo, tools and CI producing an APK. A test room, controller movement and camera, the debug overlay, and the renderer comparison.
   Done when the APK installs from Releases, the test room runs at 60 fps on the Moto G, and cloud screenshots work.
2. **M1 – Pistol.** Suppressed Five-seveN with the full feedback list (§5.3). Sprint, slide, wall-run, mantle. Locational damage, penetration, gore and ragdolls. Three enemy roles with awareness, suppression and morale. Wave mode. First finished assets: the Five-seveN, first-person arms, one enemy.
   Done when the owner wants to keep playing the wave mode.
3. **M2 – Parts system.** Loadout screen. Power, heat, mass and capacity. At least 8 passives with visible changes. The drone, whip and time slowdown.
   Done when three different loadouts play noticeably differently.
4. **M3 – Heavy weapons.** MG42-style machine gun with heat and barrel changes. Heavy armor parts, two more actives, dismemberment, machine-gunner and anti-augment enemies.
   Done when sustained fire tears up a room and the phone holds its frame rate.
5. **M4 – Look and sound.** Art direction applied to two or three test areas, lighting, an audio pass.

What comes after the prototype gets decided then.

13. Open questions

- Title.
- Does the controller hold the phone? That decides whether gyro aiming is worth building.
- How far gore goes at the maximum setting.
- Real gun names (Five-seveN, MG42, SPAS-12) are fine in a prototype. Selling the game with them might need licenses, so decide before any release.
- Budget for Meshy and ElevenLabs, if any.
- Public or private repo.